LLDD BE - Job 5 ImportImpactSaleFromIAS

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. บทนำ

1.1 วัตถุประสงค์

รับยอดขายจาก IAS + คำนวณ Growth: ทำงานเมื่อไฟล์ตอบกลับ AMS06001I มาถึง EAI S3 (`interface/out/AMS/` · มติ 2026-10-01) — EventBridge ส่ง key ของไฟล์มาเป็น `s3Key` แล้ว job อ่านไฟล์นั้นไฟล์เดียว · โฟลเดอร์นี้เป็นสองทาง (ไฟล์ขาออก AMS06001O ของ Job 4 อยู่ที่เดียวกัน) rule จึงกรองถึง prefix `AMS06001I_` และโค้ดตรวจ key ซ้ำด้วย `validateS3TriggerKey` · แทนการรับผ่าน SFTP ตามมติ 2026-08-24 บันทึกยอดขายรายวันลง sgi_sales_transactions คำนวณ sales_diff และ outlier ในหน้าต่าง 4 ช่วง × 15 วันรอบวันเปิดร้านใหม่ แล้วกำหนด sales_status = Y / N จาก growth_rate_diff

1.2 ขอบเขตงาน

- Main class/script: fgi.main.ImportImpactSaleFromIAS / FGI_ImportImpactStoreSale.sh
- Phase: B
- Output: AMS06001I (รับเข้า)
- Estimate: 13 ชั่วโมง
- Argument รับผ่าน `INPUT` (JSON) — local ใช้ env `JOB_NAME/INPUT` · AWS Batch ใช้ `argv[3]/argv[2]` · ดูหัวข้อ 5.95 · ไม่มีตาราง job_configs และไม่มีหน้าจอควบคุม (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- ตารางเวลาตั้งที่ AWS Batch scheduled event (repo ไม่มี `@Cron`) — ยกเว้น job ที่เป็น event-driven (ดูหัวข้อ Config Schema ของ job นั้น) ซึ่งห้ามตั้ง schedule · ทุก job ถูกบันทึกลง `integration_log` โดย `main.ts` อัตโนมัติ + structured log `BATCH_START/BATCH_END` พร้อม `runId`

1.3 ข้อมูลส่งมอบ

รายการ	รายละเอียด
Track	BE
Estimate	17 ชั่วโมง = implementation 13 + unit test 4 (30%)
Owner	Aphiwit <Bank> Khammoon
Legacy class เดิม	`ImportImpactSaleFromIAS` (`fcsJar/src/th/co/gosoft/fgi/main/ImportImpactSaleFromIAS.java`)
Target repository	`SBP/srm-sps-spsap-sop-sgi-batch` (NestJS 11 + TypeORM · schema `sps_store` · มติ 2026-09-02 — ย้ายมาจาก store-backend) — batch runner ของ SBP ที่รันอยู่แล้ว 42 job บน AWS Batch · ลงทะเบียน job ใน `src/main.ts` แล้วรับ argument ผ่าน `JOB_NAME/INPUT` (local) หรือ `argv[3]/argv[2]` (AWS Batch) · ไม่ผ่าน BFF และไม่เปิด HTTP · ตารางเวลาเป็น AWS Batch scheduled event ไม่ใช่ `@Cron` · ดู `srm sps spsap sop sgi batch`

1.4 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success, data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (`x-api-key` / `x-user-*`) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Job-4-PrepareImpactStoreToIAS.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Program Specification

2.1 การลงทะเบียน job และ Arguments (sop-sgi-batch)

งานนี้ลงทะเบียนเป็น job ชื่อ `sgi-import-impact-sale-from-ias` ใน `src/main.ts` ของ SBP/`srm-sps-spsap-sop-sgi-batch` โดยใช้ dispatcher เดิมของ repo (ไม่สร้าง runner ใหม่) — `main.ts` อ่านชื่อ job และ input มา 2 ทาง แล้วเลือกอัตโนมัติจากการมี `argv[3]` หรือไม่

ช่องทางรัน	ชื่อ job มาจาก	input มาจาก	ตัวอย่าง
Local / CLI / runbook	env `JOB_NAME`	env `INPUT` (JSON string)	`JOB_NAME=sgi-import-impact-sale-from-ias INPUT='{"s3Key":"interface/out/AMS/AMS06001I_20261001163030.txt"}' npm run start`
AWS Batch (ตารางเวลาจริง)	`process.argv[3]`	`process.argv[2]` (JSON string)	`node dist/main.js '{"s3Key":"interface/out/AMS/AMS06001I_202610011630.txt"}' sgi-import-impact-sale-from-ias`

ตารางเวลาไม่ได้อยู่ในโค้ด — repo นี้ไม่มี `@Cron`/`@Interval` แม้แต่จุดเดียว (แม้ติดตั้ง `@nestjs/schedule` ไว้) cron ในหัวข้อ 5 เป็น นิยามของ AWS Batch scheduled event ที่ต้องตั้งตอน deploy ไม่ใช่ค่าที่อ่านจาก config file

Argument ที่รับได้ (`INPUT` เป็น JSON object · ไม่ส่ง = `{}`)

ระบบเดิมรับ argument แบบนี้: รับ `args` แต่ ไม่ได้ใช้เลย — สแกนทุกไฟล์ในโฟลเดอร์ต้นทางที่ตรง regex `AMS06001I_YYYYMMDDHHMM.txt` (case-insensitive)
ของใหม่เปลี่ยนจาก positional string เป็น JSON เพื่อให้เพิ่มฟิลด์ได้โดยไม่พังของเดิม แต่ ต้องรองรับทุกอย่างที่ระบบเดิมรับได้เป็นอย่างน้อย — ห้ามลดความสามารถลง

Field	ชนิด	ไม่ส่งแล้วได้อะไร (default)	Validation	ตรงกับ argument เดิม
`s3Key`	string	—	key ของไฟล์ที่ EventBridge ส่งมา (กรณีปกติ) · ต้องอยู่ใต้ `interface/out/AMS/` และขึ้นต้นด้วย `AMS06001I_` · ไม่ตรง = ล้ม (`validateS3TriggerKey`)	ของใหม่ — แปลงลง `fileKey` + ตั้ง `fromS3Event=true`
`fileKey`	string	`null` = สแกนโฟลเดอร์ขาเข้าเอง	ชื่อไฟล์ · key · หรือ `s3://bucket/key` · ใช้ตอนรันมือ/rerun · ส่งชื่อไฟล์เปล่า (ไม่ใช่ JSON) ก็ได้	ของใหม่ — เจาะไฟล์เดียวเวลา rerun
`bucket`	string	`AWS_BUCKET_NAME` / `SGI_JOB5_S3_BUCKET`	บังคับ bucket · ไม่ตั้งทั้ง env และ field = `JOB5_NO_BUCKET`	ของใหม่
`dryRun`	boolean	`false`	อ่านและตรวจอย่างเดียว ไม่เขียนฐาน ไม่ย้ายไฟล์	ของใหม่
`dataName` · `sender` · `sentAt`	string	—	รับไว้เพื่อเข้ากันได้กับซองข้อความเดิม · ไม่ใช้ตัดสินเส้นทาง	ของใหม่
`dryRun`	boolean	`false`	`true` = อ่าน/คำนวณครบแต่ไม่ commit และไม่ publish/ไม่ส่งไฟล์	ของกลาง ทุก job

Field	ชนิด	ไม่ส่งแล้วได้อะไร (default)	Validation	ตรงกับ argument เดิม
`limit`	number	`null`	จำกัดจำนวนรายการที่ประมวลผลรอบนี้ (ใช้ตอน smoke test)	ของกลาง ทุก job

กติกาการ validate ที่ทุก job ต้องทำเหมือนกัน — parse `INPUT` ไม่สำเร็จ หรือฟิลด์ไม่ผ่าน validation ให้ log `BATCH_END` ด้วย `batchStatus: 'FAILED'` แล้ว `exit(1)` ก่อนแตะฐานข้อมูล (ห้าม fallback ไปค่า default เงียบ ๆ เมื่อผู้ใช้ตั้งใจส่งค่ามาแล้วผิด) · ฟิลด์ที่ไม่รู้จักให้ log warn แล้วข้าม ไม่ทำให้ job ล้ม

- • มติ 2026-10-01 — job นี้ไม่ตั้งเวลาแล้ว · ไฟล์ `AMS06001I_*.txt` มาถึง `interface/out/AMS/` → S3 Object Created → EventBridge rule → AWS Batch `SubmitJob` โดย `InputTransformer` ส่ง `{"s3Key":"<key>"}` · ชุดตั้งค่าอยู่ใน `output/aws/sgi-s3-triggers/` (สร้างด้วย `tools/build_sgi_s3_triggers.py`) · แทนแนวทางเดิมที่ consume ข้อความ RabbitMQ ของ EAI และ cron safety-net `30 16 7-16 * *`
- • `interface/out/AMS/` เป็นโฟลเดอร์สองทาง — ไฟล์ขาออก `AMS060010_*` ของ Job 4 อยู่ที่เดียวกัน rule จึงกรองถึง prefix `AMS06001I_` ไม่ใช่แค่โฟลเดอร์ และโค้ดตรวจ key ซ้ำอีกชั้น
- event ส่ง key มาเอง → job ทำไฟล์นั้นไฟล์เดียว ไม่ใช่หยิบไฟล์ล่าสุด (ไฟล์สองงวดมาใกล้กันจะไม่ตกหล่น) · ไม่ส่ง `s3Key`/`fileKey` = สแกนโฟลเดอร์เหมือนเดิม ไว้ใช้รันมือ
- การนำเข้าซ้ำถูกกันที่ระดับข้อมูลเสมอ — `checksum` + `UNIQUE(sales_summary_id, txn_date, window_no)`

2.2 Input / Output

Stage	Contract for implementation
Input	IAS sales response files from configured source path; file name pattern and pipe-delimited daily sales records.
Progress	scan files, validate pattern, parse daily sales windows, derive before/after impact metrics, write transaction rows, update working-day counts and growth status, backup processed files.
Output	FGI_IMPACT_STORE_SALES_TRN and FGI_IMPACT_STORE_SALES updated; confirm-receive rows written; source file moved to backup or error recorded.

2.3 ขั้นตอนการทำงาน

Step	Description
1	เริ่ม
2	รับ s3Key จาก S3 event → ตรวจ key → อ่านไฟล์จาก EAI S3 (WINDOWS-874) จัดกลุ่มตามร้าน + วันเปิดร้านใหม่ (EventBridge ส่ง key ของไฟล์ที่เพิ่งมาถึง (มติ 2026-10-01) · key ไม่ตรงรูป AMS06001I_ = ล้มเสียงดัง)
3	เป็นงวดที่ยังไม่นำเข้า? | No: จบ (idempotency guard กันนำเข้าซ้ำ)
4	เปิด transaction ต่อไฟล์ แล้ว insert sgi_sales_transactions แถวดิบ (ระวัง: catch ใน DAO บางจุดอาจทำให้ rollback ไม่ทำงาน)
5	insert sgi_interface_transactions: data_name = IMPACT_STORE_SALES · direction = IN · status = COMPLETED (บันทึก*การรับไฟล์* — ทำทันทีที่อ่านไฟล์สำเร็จ ไม่ผูกกับผล Y/N เพราะไฟล์มาถึงแล้วไม่ว่าผลจะเป็นอะไร (ถ้าผูกกับสาขา Y งวดที่ผลเป็น N จะไม่มีบันทึก แล้ว Job 10 จะเข้าใจว่า IAS ไม่ตอบกลับ) · เป็นขารับกลับของรอบที่ Job 4 ส่งออก จึงใช้ data_name เดิม — UNIQUE (data_name, direction, business_key, period_key) แยกขา OUT/IN ให้อยู่แล้ว · typed FK = sales_summary_id · acked_at = now)
6	total_working_days = จำนวนแถวดิบทั้งหมด (นับรวมแถวนอกหน้าต่างคำนวณด้วย (raw count))
7	ต้องคำนวณ sales_diff? (ไม่เข้าเงื่อนไข pre-accept) | No: Pre-accept: sales_status = Y ทันที (pre-accept เมื่ออายุร้าน < 12ด.15ว. หรือวันทำการ < 60)

Step	Description
8	คำนวณ sales_diff รายวัน + outlier แบบจับคู่ (|sales_diff| ≥ 50) (4 หน้าต่าง × 15 วัน ไม่รวมวันเปิดร้านใหม่ / ธงรวมอดีต-ปัจจุบันต้องตรงกัน)
9	COALESCE(growth_rate_diff, −1) < 0 ? | No: sales_status = N (ไม่เข้าเกณฑ์ชดเชย) (NULL ถูกแทนด้วย −1 = accept อัตโนมัติ (ความเสี่ยง P1))
10	sales_status = Y (เข้าเกณฑ์ชดเชย) (ผลทางธุรกิจอย่างเดียว — การรับไฟล์บันทึกไปแล้วตั้งแต่ต้น ไม่ขึ้นกับผล Y/N)
11	ย้ายอ็อบเจกต์ไป interface/backup-out/AMS/ (ย้ายเสมอเมื่อนำเข้าสำเร็จ (มติ 2026-10-15 · ไม่มีสวิตช์ env) · ยังต้องผ่าน canArchiveFile() ก่อนเสมอ)
12	จบ

2.4 เงื่อนไขตัดสิน (Decision Rules) — ตัดสินจากอะไร

Job 5 ตัดสิน 3 เรื่อง: ไฟล์ไหนควรอ่าน · บรรทัดไหนจับคู่กับร้านได้ · และ ยอดขายครบพอจะคำนวณ Growth หรือยัง

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
ไฟล์นี้ควรอ่านหรือไม่	ชื่ออ็อบเจกต์บน EAI S3 — `interface/out/AMS/` (● แก้ 2026-09-30 · ยืนยันกับ bucket จริง: ไฟล์ `AMS06001I_20260924145 2.txt` อยู่ที่นั่น · เดิมตั้งค่าไว้ที่ `interface/in/IAS/` ซึ่งว่างเปล่า) · โฟลเดอร์นี้ใช้รวมสองทาง มีไฟล์ `AMS06001O_*` ที่ Job 4 อัปโหลดปนอยู่ จึงต้องคัดด้วย prefix ชื่อไฟล์เสมอ	ต้องตรงรูปแบบ `AMS06001I_YYYYMMDDHHMM.txt` (ตรวจด้วย regex เดิม · case-insensitive) — ไม่ตรง = ข้ามไฟล์นั้น	ระบบเดิม สแกนทุกไฟล์ในโฟลเดอร์ ไม่ได้ใช้ argument เลย · ของใหม่ทำงานตาม `s3Key` ที่ S3 event ส่งมา (ไฟล์เดียว · มติ 2026-10-01) และรับ `fileKey` เพื่อเจาะไฟล์เดียวตอนรันมือ · key จาก event ที่ไม่ตรงรูปแบบ = ล้ม ไม่ใช่ข้ามเงียบ ๆ
ไฟล์นี้เคยประมวลผลไปแล้วหรือยัง	`sgi_interface_transactions` — `data_name = 'IMPACT_STORE_SALES'` · `direction = 'IN'` · `file_name` · `file_checksum`	มี transaction ของ `file_name` เดิมและ checksum ตรงกัน = เคยอ่านแล้ว → ข้าม	อ่านสำเร็จจึงย้ายอ็อบเจกต์ไป `interface/backup-out/AMS/` — ย้ายไฟล์ต้องเป็นขั้นสุดท้าย ไม่ใช่ก่อน commit · ย้ายเสมอเมื่อนำเข้าสำเร็จ (มติ 2026-10-15 · ไม่มีสวิตช์ env) โดยต้องผ่าน `canArchiveFile()` ที่ตัดสินว่าผลการนำเข้าสะอาดพอจะย้ายได้ไหม
บรรทัดนี้จับคู่กับร้านไหน	`sgi_fgi_impact_stores` (`impacted_store_code` + วันเปิดร้านใหม่) — ไฟล์มี 4 ฟิลด์: impacted_store_code | วันเปิดร้านใหม่ | วันที่ขาย | ยอดขาย	จับคู่ด้วย `impacted_store_code` + วันเปิดร้านใหม่ (ตรงกับคีย์ที่ Job 4 ส่งออกไป) · จับคู่ไม่ได้ = reject	reject รายแถวต้องไม่ทำให้ทั้งไฟล์ fail — สรุปจำนวนใน metrics และแนบไปในอีเมลแจ้งผล
ยอดขายครบพอคำนวณ Growth หรือยัง · pre-accept	`sgi_sales_transactions` (ยอดรายวัน) → สรุปลง `sgi_fgi_impact_sales_summaries` (`total_working_days` · `growth_rate_before/after/diff` · `sales_status`) + วันเปิดร้านจาก `iasRepresentativeCte()` (ชุดเดียวกับ Job 4)	● แก้ 2026-10-05 — ข้อสรุปเดิมของหัวข้อนี้ผิด (ตรวจโค้ด Java ซ้ำแล้ว): ระบบเดิม `ImportService.java:116` เรียก `ImportJdbc.updateStatusImpactStoreSalesByDays` (136-153) ก่อน คำนวณ growth — ตั้ง `FLAG_VERIFY = 'Y'` ทันทีเมื่อ `OPENDATE_I > ADD_MONTHS(OPENDATE_N,-12)-15` หรือ `TOTAL_WORKING_DAY != 60` · แล้ว `updateStatusImpactStoreSaleByDiff` (155-175) แตะเฉพาะแถวที่ยังเป็น P → แถว pre-accept คง Y · growth ยังคำนวณให้ทุกแถว (หน้าจอแสดงคานี้)	pre-accept → Y ทันที (มติผู้ใช้ 2026-10-05: ยึด `!= 60` ตามระบบเดิม · `SGI_JOB5_PREACCEPT_NEQ`) · นอกนั้นตัดสินจาก growth (`< 0` = Y · NULL = Y ตามระบบเดิม · `SGI_JOB5_NULL_DIFF_STATUS`) · โค้ด `decideFinalSalesStatus()` ใน `job-5-growth-rate.ts` · ▲ โค้ดก่อน 2026-10-05 แค่นับ pre-accept แล้วตัดสินจาก growth อย่างเดียว → ร้านวันทำการ 45 วันที่ growth บวก ระบบเดิมจ่ายแต่ของใหม่ให้ N (แก้แล้ว) · หน้าจอรายการใช้ `< 60` ตีแถวแดง ซึ่งเป็นคนละกติกากับ pre-accept มาแต่เดิม

ค่าคงที่และโดเมนที่ใช้ในเงื่อนไขข้างบน

ทุกค่าในตารางนี้ต้องอ่านจาก config/env ไม่ hardcode ในคิวรี — แต่ ค่าตั้งต้นต้องเท่าระบบเดิมทุกตัว ไม่งั้นผลลัพธ์จะไม่ตรงกันตอนเทียบ UAT

ค่า	โดเมน / ค่าที่ระบบเดิมใช้	ที่มา
รูปแบบชื่อไฟล์	`AMS06001I_YYYYMMDDHHMM.txt`	regex ใน `ImportController.importImpactSaleFromIAS`
encoding ของไฟล์	ประกาศเป็น WINDOWS-874 แต่ไฟล์จริง เป็น ASCII ล้วน ไม่มีไบต์ภาษาไทยเลย · วันที่ในไฟล์เป็น ค.ศ. (`20260824`) ไม่ใช่ พ.ศ.	• แก้ 2026-09-16 จากไฟล์ตัวอย่างจริง `docs/file_IAS_STA/` — เอกสารเดิมเขียนว่าเป็น พ.ศ. ซึ่งผิด · ที่เป็น พ.ศ. + windows-874 จริงคือ `FRBC0001` ของ STA (Job 6) เท่านั้น · รายละเอียด `IAS STA interface files`
จำนวนวันทำการขั้นต่ำ	60 วัน	`FgiConstant.TOTAL_INTERVAL_DAY = 60`
`sales_status`	`W` = รอ · `P` = ส่งคำขอแล้วรอผล (Job 4 ตั้ง) · `Y` = เข้าเกณฑ์ชดเชย · `N` = ไม่เข้าเกณฑ์ · `E` = ผิดพลาด	`CHECK` ใน DDL · Job 5 แตะเฉพาะแถวที่เป็น `'P'` เท่านั้น (กันทับผลที่คนแก้ไปแล้ว)
`sales_diff` ของแถวรายวัน	เปอร์เซ็นต์ = ROUND((ยอดวันนั้น − ค่าเฉลี่ยของหน้าต่าง) / ยอดวันนั้น × 100)	`ImportJdbc.java:544` · ▲ หารด้วยยอดของวันนั้นเอง ไม่ใช่หารด้วยค่าเฉลี่ย — หน่วยเป็น % ไม่ใช่บาท
`is_outlier`	`|sales_diff| >= 50` (เปอร์เซ็นต์)	`ImportJdbc.java` — `CASE WHEN ABS(T.DIFF_*) >= 50 THEN 1 ELSE 0 END`
ตัวกรองจับคู่วันตอนเฉลี่ย	ถ้าหน้าต่างปีก่อนกับปีนี้ มี outlier ไม่เหมือนกัน วันที่ธง outlier ไม่ตรงกันจะถูกแทนด้วย 0 แต่ยังนับอยู่ในตัวหาร (ตัวหารคงเป็น 15)	`ELSE 0` ในสูตร AVG ของเดิม — ไม่ใช่การตัดวันนั้นทิ้ง · เขียนเป็น `WHERE` แล้วผลจะต่างทันที · จับคู่ด้วย `seq` (ตารางเดิมเป็นตารางกว้างจึงจับคู่ได้ในแถวเดียว ตารางใหม่แคบจึงต้องมีคอลัมน์ `seq`)

2.5 Field, Format, and Validation

Field / UI	Format	Validation	Behavior
ตัวกระตุ้น (Trigger)	S3 Object Created → EventBridge rule → AWS Batch SubmitJob	แก้ไขได้	มติ 2026-10-01 · rule กรอง `interface/out/AMS/AMS06001I_` (ไม่ใช่แค่โฟลเดอร์ ไม่งั้นไฟล์ขาออกของ Job 4 จะปลุก Job 5) · INPUT: `{"s3Key":"…"}` → แปลงเป็น fileKey + fromS3Event · ฟิลด์อื่นที่รับ: fileKey · bucket · dataName · sender · sentAt · dryRun
ไฟล์ขาเข้า (EAI S3)	interface/out/AMS/AMS06001I_yyyyMMddHHmm.txt (WINDOWS-874, 4 ฟิลด์)	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	impacted_store_code | วันเปิดร้านใหม่ | วันที่ขาย | ยอดขาย (4 ฟิลด์ตามสัญญาไฟล์ของ IAS)
หน้าต่างคำนวณ	4 ช่วง × 15 วัน รอบวันเปิดร้านใหม่ (ไม่รวมวันเปิด) — วันเปิดร้านอ่านจาก master ของระบบ SBP เดิม	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	
เกณฑ์ Outlier	|sales_diff| ≥ 50	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	literal ในโค้ด — เปลี่ยนต้องอนุมัติธุรกิจ (8.2)
วันทำการคาดหวัง	60	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	ถ้าไม่เท่า 60 → pre-accept เป็น Y ทันที
กฎ Pre-accept	อายุร้าน < 12 เดือน 15 วัน หรือวันทำการ < 60 → Y	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	
ย้ายไฟล์เข้า backup	ย้ายเสมอ (ไม่มีสวิตช์ env · มติ 2026-10-15)	แก้ไขได้	ปลายทาง `interface/backup-out/AMS/` · ไม่ยกเลิก canArchiveFile() ที่ตัดสินว่าผลการนำเข้าสะอาดพอจะย้ายได้ไหม

2.6 หลักฐานที่ต้องเก็บตอนรัน

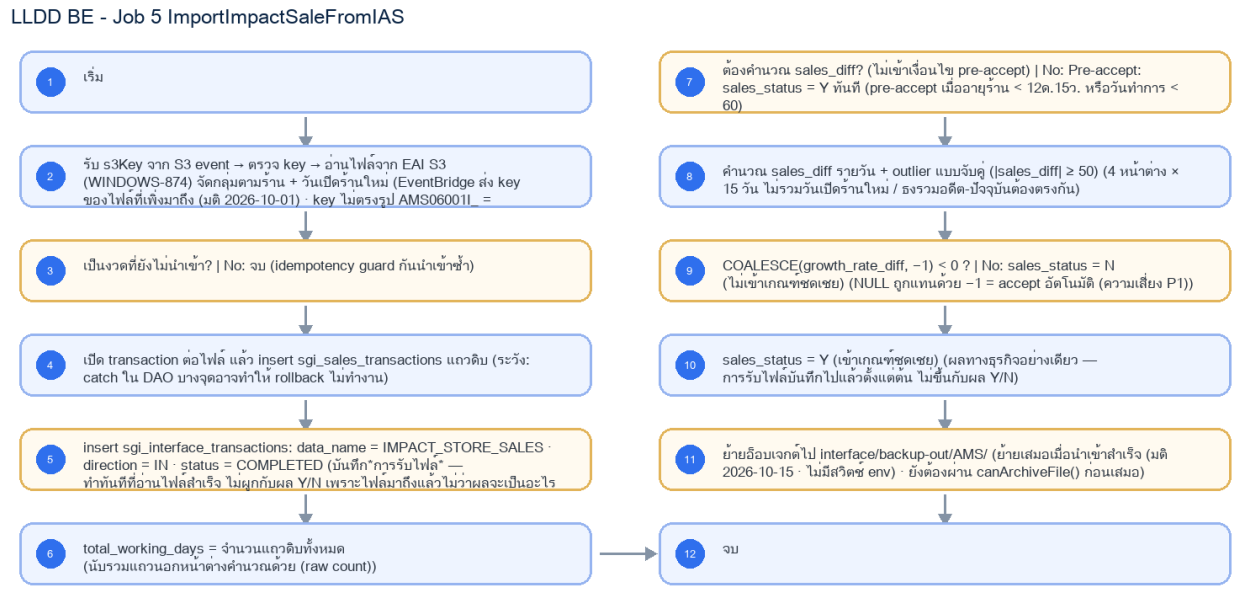
Evidence	Job-specific value	Acceptance
Input identity	IAS sales response files from configured source path; file name pattern and pipe-delimited daily sales records.	บันทึกไฟล์/คีย์ธุรกิจ/งวดที่รับเข้าใน run record
Output identity	FGI_IMPACT_STORE_SALES_TRN and FGI_IMPACT_STORE_SALES updated; confirm-receive rows written; source file moved to backup or error recorded.	กระทบยอด input · สำเร็จ · ตก · ข้าม
Dedup proof	checksum กันไฟล์ซ้ำ + UNIQUE(sales_summary_id,txn_date,window_no); คำนวณ summary ใหม่จาก transaction rows ทุก rerun	รันซ้ำด้วย fixture เดิมต้องไม่เกิดคีย์ธุรกิจซ้ำ
Transaction proof	upsert รายวันและ update summary ของ sales_summary_id เดียวกันใน transaction; checksum/file tracking commit พร้อมกัน	จำลอง error กลางทางแล้วไม่เหลือแถวค้างนอกขอบเขตที่ระบุ
Security proof	สิทธิ์อ่าน EAI S3 ใช้ IAM role ของ AWS Batch หรือ secretRef=secret/sgi/interfaces/eai-s3 จำกัดเฉพาะ interface/out/AMS/ (GetObject) + interface/backup-out/AMS/ (PutObject เฉพาะ backup); quarantine อ็อบเจกต์ที่ checksum/รูปแบบไม่ผ่าน แทนการลบทิ้ง	config/log/error ไม่มีความลับแบบข้อความธรรมดา

2.7 โค้ด Java เดิมที่อ้างอิง

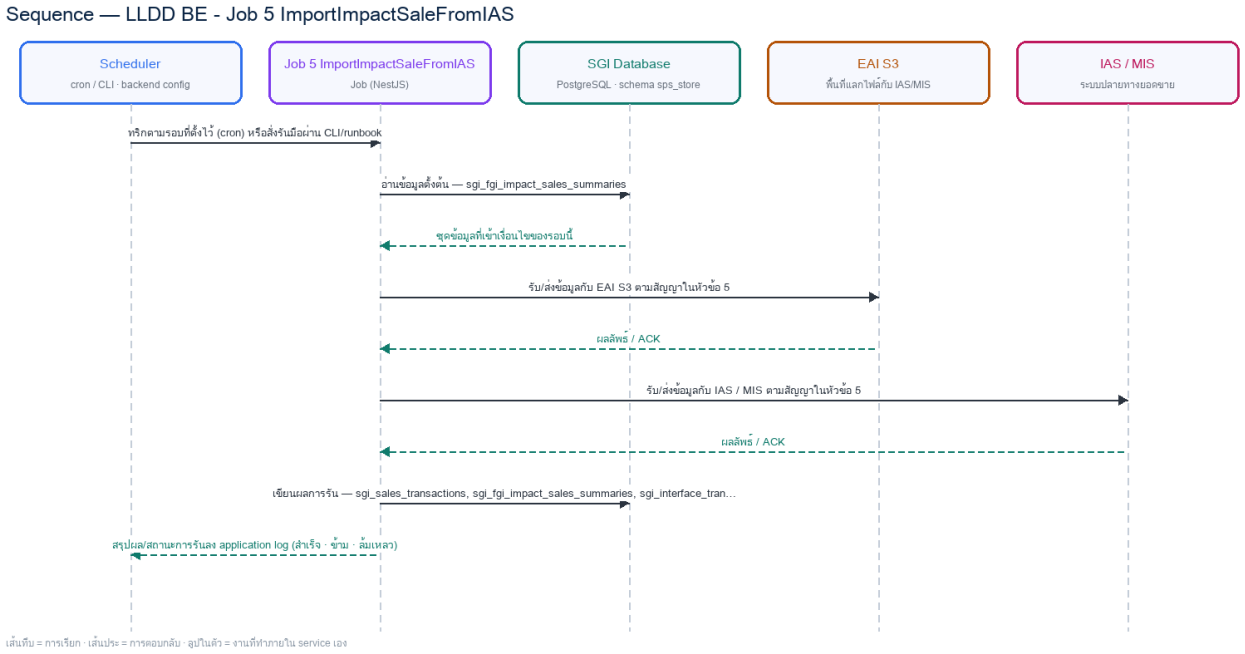
Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/ImportImpactSaleFromIAS.java	9-19	Legacy main entrypoint that delegates to import controller.
fcsJar/src/th/co/gosoft/fgi/controller/ImportController.java	101-411	Parse IAS file, compute sales windows, prepare inserts/updates, backup and notify.
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportJdbc.java	136-182, 517-804	Update verification flags, working days, growth-rate calculations, cleanup old files.

Line ranges refer to the legacy Java implementation under `batchjob/fcsJar/` (path นับจากราก `sbp-prototype/`). Use these ranges to preserve business behavior while implementing the target Node job.

3. Flow และ Sequence Diagram



รูปที่ 1: Implementation flow: LLDD BE - Job 5 ImportImpactSaleFromIAS



รูปที่ 2: Sequence diagram: LLDD BE - Job 5 ImportImpactSaleFromIAS

4. Reference DB Mapping

อ้างอิงสำหรับ implement job เท่านั้น ไม่ใช่งานออกแบบ/สร้างหน้า Database

Table / Object	R/W	Usage
sgi_sales_transactions	W	ยอดขายรายวันดิบจากไฟล์ (4 หน้าต่างเวลา)
sgi_fgi_impact_sales_summaries	R/W	อัปเดต total_working_days, growth_rate_diff, sales_status Y/N
sgi_interface_transactions	W	tracking: data_name=IMPACT_STORE_SALES · direction=IN · status=COMPLETED (ขารับกลับของรอบที่ Job 4 ส่งออก) · typed FK = sales_summary_id

5. โค้ดจริงอยู่ที่ไหน

เอกสารนี้ ไม่วาง skeleton code — job ทั้ง 11 ตัว implement แล้ว การมีโค้ดสองชุด (ในเอกสาร + ของจริง) ทำให้หลุดจากกันแน่นอน · อ่านสัญญาที่หัวข้อ 2 แล้วดูของจริงตามตารางนี้

สิ่งที่ต้องดู	ที่อยู่
ชื่อ job ที่ลงทะเบียน	`sgi-job-5` ใน `SBP/srm-sps-spsap-sop-sgi-batch/src/main.ts`
service ของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-5-*.service.ts`
เทสของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-5-*.spec.ts`
SQL ที่ใช้จริง	อยู่ในไฟล์ service เดียวกัน (repo นี้ไม่แยก `*.sql.ts`)
คำอธิบายภาษาคนของ job	batchjob/JOB-05-* อธิบายละเอียด

6. Acceptance Criteria

- พารามิเตอร์รับผ่าน `INPUT` (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ `@Cron` ในโค้ด
- การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนพึ่ง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้
- ทุกรอบต้องเขียน application log แบบ structured (BATCH_START/BATCH_END + `runId` — `src/main.ts` ทำให้แล้ว) และบันทึกลง `integration_log` อัตโนมัติ · error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

7. Unit Test Case

38 เคส · ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง (mock ที่ขอบ repository/client) · ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่ · ● เทสที่ mock query runner ต้องป้อนรูป tuple `[rows, rowCount]` สำหรับ UPDATE/DELETE (กฎ #140)

การค้นหาและอ่านไฟล์จาก S3

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-01	มีหลายไฟล์ตรง pattern บน source prefix	ไฟล์ `AMS06001I_202605171530.txt`, `AMS06001I_202605171545.txt`	ได้ทั้งสองไฟล์เข้าลิสต์ที่ต้องประมวลผล
UT-02	ต้อง list ไฟล์หลายหน้า	S3 คืนผลแบบแบ่งหน้า 2 หน้า	รวมไฟล์จากทุกหน้าก่อนกรอง pattern
UT-03	ไม่มีไฟล์ตรง pattern เลย	source prefix ไม่มีไฟล์ที่ตรงชื่อ	สรุปผลเป็น NO_DATA พร้อม note
UT-04	มีไฟล์ที่ชื่อไม่ตรง pattern ปนอยู่	ไฟล์ที่ชื่อไม่ตรง regex ปนกับไฟล์ที่ตรง	เลือกเฉพาะไฟล์ที่ตรง pattern เท่านั้น ไม่นำไฟล์อื่นมาประมวลผล
UT-05	อ่านไฟล์ด้วย encoding windows-874	ไฟล์ที่มีตัวอักษรไทยใน encoding นี้	อ่านเนื้อหาได้ถูกต้อง ไม่เพี้ยน
UT-06	parse ไฟล์ไม่สำเร็จ (เช่นวันที่ผิดรูปแบบ)	แถวที่คอลัมน์วันที่เปิดร้านใหม่ไม่ใช่ yyyyMMdd	throw error ทันที ทำให้ทั้ง batch หยุด ไม่ประมวลผลไฟล์ถัดไป

การจัดกลุ่มและคำนวณจำนวนวันทำการ

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-07	ข้อมูลของคู่รหัสร้าน+วันเปิดร้านใหม่เดียวกันเรียงติดกันในไฟล์	5 แถวติดกันของคู่เดียวกัน	นับจำนวนวันทำการได้ 5 และรวมเป็นกลุ่มเดียว
UT-08	ข้อมูลเปลี่ยนคู่รหัสร้านหรือวันเปิดร้านใหม่กลางไฟล์	แถว 1-3 เป็นคู่ A, แถว 4-6 เป็นคู่ B	ปิดกลุ่ม A ด้วยจำนวนวันทำการ 3 ก่อนเริ่มนับกลุ่ม B ใหม่จาก 0
UT-09	ปิดกลุ่มสุดท้ายของไฟล์	แถวสุดท้ายของไฟล์เป็นคู่ที่ยังไม่ปิดกลุ่ม	บันทึกจำนวนวันทำการของกลุ่มสุดท้ายและย้ายเข้ารายการที่จะ insert ด้วย

การกันข้อมูลซ้ำ

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-10	คู่รหัสร้าน+เดือน+ปีมีข้อมูลอยู่แล้วในตารางยอดขายที่ประมวลผลแล้ว	แถวที่ตรงกับ record ที่มีอยู่แล้ว	ข้ามทั้งแถวนั้น ไม่นำไปคำนวณ ไม่เก็บเข้ากลุ่ม
UT-11	คู่รหัสร้าน+เดือน+ปียังไม่มีข้อมูล	แถวใหม่	นำไปคำนวณและเก็บเข้ากลุ่มตามปกติ
UT-12	record ประวัติซ้ำกัน (รหัสอ้างอิงธุรกรรม+ปี+เดือนเดียวกัน) ภายในไฟล์เดียวกัน	2 record ประวัติที่ค่าตรงกันทั้งสามฟิลด์	เก็บเข้ารายการประวัติเพียงรายการแรก ข้ามรายการที่ซ้ำ

การคำนวณลำดับอ้างอิงวัน (seq) และยอดขายตามช่วง

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-13	วันที่มียอดขายอยู่ในช่วงก่อนกระทบ 15 วัน ของปีก่อนหน้า	วันขาย = วันเปิดร้านใหม่ - 1 ปี - 10 วัน	กำหนดค่าเข้า `date_before_impact_past`/`sales_before_impact_past`
UT-14	วันที่มียอดขายอยู่ในช่วงหลังกระทบ 15 วัน ของปีก่อนหน้า	วันขาย = วันเปิดร้านใหม่ - 1 ปี + 10 วัน	กำหนดค่าเข้า `date_after_impact_past`/`sales_after_impact_past`
UT-15	วันที่มียอดขายอยู่ในช่วงก่อนกระทบ 15 วัน ของปีปัจจุบัน	วันขาย = วันเปิดร้านใหม่ - 10 วัน	กำหนดค่าเข้า `date_before_impact_present`/`sales_before_impact_present`
UT-16	วันที่มียอดขายอยู่ในช่วงหลังกระทบ 15 วัน ของปีปัจจุบัน	วันขาย = วันเปิดร้านใหม่ + 10 วัน	กำหนดค่าเข้า `date_after_impact_present`/`sales_after_impact_present`
UT-17	วันเปิดร้านใหม่ตรงกับวันที่ 28 กุมภาพันธ์และปีก่อนหน้าเป็นปีอธิกสุรทิน	วันเปิดร้านใหม่ = 28/02/2028, ปีก่อนหน้า (2027) ไม่ใช่ปีอธิกสุรทิน แต่คำนวณยอนไปตรง 29/02	ปรับวันที่อ้างอิงของปีก่อนหน้าให้ใช้ 29 กุมภาพันธ์ตามการปรับพิเศษในโค้ดเดิม
UT-18	วันที่มียอดขายไม่ตรงกับช่วงใดเลยทั้ง 4 ช่วง	วันขายที่ห่างจากวันเปิดร้านใหม่เกิน 15 วันทุกกรณี	record ยังถูกเก็บแต่ทุก field ยอดขาย/วันที่เป็นค่าวางเปล่า
UT-19	ยอดขายของแถวเป็น 0	ยอดขาย = 0 ในคอลัมน์ที่ตรงกับช่วงใดช่วงหนึ่ง	เมื่อ insert ค่ายอดขายในคอลัมน์นั้นเป็นค่าว่างเปล่า (ไม่ insert เป็นเลข 0)

การบันทึกข้อมูล / Transaction

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-20	ไม่มีรายการที่จะ insert เลย (ทุกแถวถูกข้ามเพราะซ้ำ)	ไฟล์ที่ทุกคู่รหัสร้าน+เดือน+ปีมีข้อมูลอยู่แล้ว	ไม่เปิด transaction, ไม่ backup ไฟล์, ข้ามไปไฟล์ถัดไป
UT-21	insert ยอดขาย, update จำนวนวันทำการ, update สถานะ, คำนวณ growth rate, คำนวณ diff, update สถานะสุดท้าย, insert ประวัติ สำเร็จทั้งหมด	รายการที่จะ insert ไม่ว่าง	commit transaction, ทำ backup ไฟล์ต่อ

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-22	เกิด error ระหว่างงานใดๆในทรานแซกชัน	mock DB error ที่ขั้นตอนคำนวณ growth rate	rollback ทั้งหมดในรอบไฟล์นี้ (รวม insert ยอดขายที่ทำไปก่อนหน้า), หยุดทั้ง batch ไม่ประมวลผลไฟล์ถัดไป
UT-23	update สถานะเป็นผ่านตามเกณฑ์วันเปิด/จำนวนวันทำการ	คู่ที่ร้านเดิมเปิดไม่ครบ 12 เดือน 15 วันก่อนร้านใหม่	เปลี่ยนสถานะเป็นผ่าน (Y) โดยไม่ต้องรอครบ 60 วันทำการ
UT-24	update สถานะเป็นผ่านตามเกณฑ์จำนวนวันทำการยังไม่ครบ 60	คู่ที่ `total_working_day <> 60`	เปลี่ยนสถานะเป็นผ่าน (Y)
UT-25	คู่ที่ผลต่างอัตราการเติบโตติดลบ	`growth_rate_diff < 0`	สถานะสุดท้ายเป็น `Y`
UT-26	คู่ที่ผลต่างอัตราการเติบโตไม่ติดลบหรือไม่มีค่า	`growth_rate_diff >= 0` หรือเป็นค่าว่างเปล่า	สถานะสุดท้ายเป็น `N`

การสำรองไฟล์บน S3

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-27	copy ไป backup prefix สำเร็จหลัง commit transaction	transaction ของไฟล์นี้ commit สำเร็จ	copyObject สำเร็จ, บันทึกสถานะไฟล์นี้เป็นสำเร็จ
UT-28	copy ไป backup prefix ล้มเหลว	mock S3 copyObject ล้มเหลวหลัง commit สำเร็จ	บันทึกสถานะไฟล์นี้เป็นล้มเหลว แต่ข้อมูลที่ commit ไปแล้วในฐานข้อมูลไม่ถูก rollback

การลบไฟล์ export เก่าข้าม job

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-29	มีไฟล์ export ที่อายุครบ 1 เดือนและสถานะเป็นผ่าน/ไม่ผ่านแล้ว	คู่รหัสร้าน+เดือน+ปีที่ `flag_verify IN ('Y','N')` และอยู่ในงวดของเดือนก่อนหน้า	ได้รายชื่อไฟล์ export ที่ต้องลบ แล้วลบไฟล์เหล่านั้นจาก output prefix ของ job ฝั่งส่งออก
UT-30	ไม่มีไฟล์ที่ต้องลบ	query คืนลิสต์ว่าง	ข้ามขั้นตอนลบไฟล์ ไม่มี error
UT-31	ไฟล์ที่ต้องลบไม่มีอยู่จริงบน S3 แล้ว (ถูกลบไปก่อนหน้า)	mock deleteObject คืน NotFound	ถือว่าข้ามได้ ไม่ throw error

ผลลัพธ์มาตรฐานและ Error

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-32	เกิด error กลางทาง (เช่น ระหว่างประมวลผลไฟล์ที่ 2 จาก 3 ไฟล์)	mock error ที่ไฟล์ที่สอง	ผลลัพธ์ FAILED ยังคงรายชื่อไฟล์ที่ประมวลผลสำเร็จไปแล้ว (ไฟล์แรก) ไว้ครบ ไม่ประมวลผลไฟล์ที่สาม
UT-33	error message ที่แสดงใน result ต้อง sanitize	error ที่มี connection string/credential อยู่ใน stack trace เดิม	ข้อความ error ที่แสดงไม่มีข้อมูลอ่อนไหวหลุดออกมา
UT-34	ผลลัพธ์เป็นสำเร็จหรือไม่มีข้อมูล	ทำงานจบปกติ	exit code = `0`
UT-35	ผลลัพธ์เป็นล้มเหลว	เกิด error ณ จุดใดก็ตาม	exit code = `1`

การแจ้งเตือน / Notification

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-36	หา template และส่งอีเมลสำเร็จ	mock ทั้งสองขั้นตอนสำเร็จ	อีเมลถูกส่งด้วยเนื้อความที่ render จากผลลัพธ์จริง ไปยังผู้รับที่ตั้งค่าไว้ใน config
UT-37	หา template ไม่พบ	mock query template คืนค่าว่าง	log ไว้เท่านั้น ไม่ throw error, ไม่กระทบผลลัพธ์ที่ commit แล้ว
UT-38	ส่งอีเมลผ่าน SMTP ล้มเหลว	mock SMTP ล้มเหลว	log ไว้เท่านั้น, ผลลัพธ์ที่ได้ก่อนหน้าไม่เปลี่ยนเป็นล้มเหลว, ไม่มีการ retry

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. ImportImpactSaleFromIAS.java — lines 9-19

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ImportImpactSaleFromIAS.java
Original lines	9-19
Responsibility	Legacy main entrypoint that delegates to import controller.

```
public class ImportImpactSaleFromIAS {
private static ApplicationContext context = new FileSystemXmlApplicationContext("config.xml");
private static ImportController importController = (ImportController) context.getBean("importController");

public static void main(String[] args) {
LogUtils.info(ImportImpactSaleFromIAS.class,"============****** Start => import FGI_IMPACT_STORE_SALES_TRN from MIS ******============");
importController.importImpactSaleFromIAS( args );
LogUtils.info(ImportImpactSaleFromIAS.class,"============****** End => import FGI_IMPACT_STORE_SALES_TRN from MIS ******============");
}

}
```

J2. ImportController.java — lines 101-112

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ImportController.java
Original lines	101-112
Responsibility	Parse IAS file, compute sales windows, prepare inserts/updates, backup and notify.

```
public void importImpactSaleFromIAS(String[] args){
configFtp = (FtpBean)context.getBean("fgiImportImpactSaleFromIAS");
FtpBean configExportFtp = (FtpBean)context.getBean("fgiPrepareImpactStoreToIAS");
setValueConfig();
EmailTemplateInformStatusBean informBean = new EmailTemplateInformStatusBean();
Calendar cal = new GregorianCalendar();
boolean isFile = false;
FileInputStream openFileMsgProp = null;

try {
cal = new GregorianCalendar();
int dayStart = cal.get(Calendar.DATE);
```

J2. ImportController.java — lines 400-411

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ImportController.java
Original lines	400-411
Responsibility	Parse IAS file, compute sales windows, prepare inserts/updates, backup and notify.

```
if(openFileMsgProp!=null){
th.co.gosoft.fcs.utils.FileUtils.closeFile(openFileMsgProp);
}
} catch (IOException ex) {
LogUtils.info(getClass(),"Error Close File Prop : "+ex);
informBean.setErrorMsg(""+ex);
informBean.setExportFileName("");
informBean.setNote("Error Close File Prop ");
}
}

        clearValueConfig();
```

J3. ImportJdbc.java — lines 136-147

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportJdbc.java
Original lines	136-147
Responsibility	Update verification flags, working days, growth-rate calculations, cleanup old files.

```
public int updateStatusImpactStoreSalesByDays() {
StringBuffer sql = new StringBuffer();
int result = 0;
try{
sql.append(" UPDATE FGI_IMPACT_STORE_SALES SET ");
sql.append(" FLAG_VERIFY = '" + FgiConstant.FLAG_VERIFY_ACCEPT + "', ");
sql.append(" UPDATE_DATE = SYSDATE ");
sql.append(" WHERE FLAG_VERIFY in ( '" + FgiConstant.FLAG_VERIFY_WAIT + "','" + FgiConstant.FLAG_VERIFY_ON_PROCESS + "') ");
sql.append(" AND ( ");
sql.append(" OPENDATE_I > TRUNC(ADD_MONTHS(OPENDATE_N, '-" + FgiConstant.INTERVAL_MONTH + "')-'" + FgiConstant.INTERVAL_DAY + "') ");
sql.append(" OR TOTAL_WORKING_DAY != '" + FgiConstant.TOTAL_INTERVAL_DAY + "' ");
sql.append(" ) ");
```

J3. ImportJdbc.java — lines 171-182

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportJdbc.java
Original lines	171-182
Responsibility	Update verification flags, working days, growth-rate calculations, cleanup old files.

```
sql.append(" FROM FGI_IMPACT_STORE_SALES_TRN TRN ");
sql.append(" WHERE TRN.STORECODE_I = FISS.STORECODE_I ");
sql.append(" AND TRN.MONTH = FISS.MONTH ");
sql.append(" AND TRN.YEAR = FISS.YEAR ");
sql.append(" ) ");
result = jdbcTemplate.update(sql.toString());
LogUtils.info(getClass(),"----- update FGI_IMPACT_STORE_SALES (FLAG_VERIFY=Y/N) : " + result + " -----");
}catch(Exception e){
LogUtils.error(getClass(),e);
}
return result;
}
```

J3. ImportJdbc.java — lines 517-528

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportJdbc.java
Original lines	517-528
Responsibility	Update verification flags, working days, growth-rate calculations, cleanup old files.

```
public int[] updateTotalWorkingDayImpactStoreSales(List impactSaleList) {
StringBuffer sql = new StringBuffer();
try{
sql.append(" UPDATE FGI_IMPACT_STORE_SALES SET ");
sql.append(" TOTAL_WORKING_DAY = ? ");
sql.append(" WHERE FLAG_VERIFY = '" + FgiConstant.FLAG_VERIFY_ON_PROCESS + "' ");
sql.append(" AND STORECODE_I = ? ");
sql.append(" AND MONTH = ? ");
sql.append(" AND YEAR = ? ");
int[] result = jdbcTemplate.batchUpdate(sql.toString(), new UpdateTotalWorkingDayImpactStoreSalesFromIASBatch(impactSaleList));
LogUtils.info(getClass(),"----- update FGI_IMPACT_STORE_SALES : " + result.length + " -----");
return result;
```

J3. ImportJdbc.java — lines 793-804

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportJdbc.java
Original lines	793-804
Responsibility	Update verification flags, working days, growth-rate calculations, cleanup old files.

```
sql.append(" and to_date(rd.year || rd.month, 'YYYYMM') = trunc(add_months(sysdate, -1), 'MM') ");
sql.append(" where fiss.flag_verify in ('Y', 'N') ");
return jdbcTemplate.query(sql.toString(), new RowMapper() {

@Override
public Object mapRow(ResultSet rs, int index) throws SQLException {
String result = rs.getString("INTERFACE_TYPE")+FILE_EXTENSION;
return result;
}
});
}
}
```